AIVA: AI Vision Assistant

Technical Project Report & System Architecture
Hackathon Submission | February 2026

1. Executive Summary & Abstract

AIVA (AI Vision Assistant) is a groundbreaking assistive technology platform designed to restore spatial autonomy to visually impaired individuals. Addressing the critical gap between expensive, proprietary hardware and basic smartphone apps, AIVA leverages a **hybrid edge-cloud architecture** to deliver real-time environmental perception.

By fusing Computer Vision (YOLOv8, MiDaS) and Generative AI (Google Gemini), AIVA converts visual data into actionable audio insights. The system provides instantaneous obstacle warnings, reads text via OCR, identifies people, and offers rich scene descriptions—all through a low-latency Android interface. This project demonstrates that high-fidelity assistive AI can be accessible, scalable, and privacy-preserving.

2. Problem Statement & Objectives

The Societal Challenge:

Over 2.2 billion people globally suffer from vision impairment. Current solutions force a compromise: specialized hardware is cost-prohibitive ($3,000+), while mobile apps often suffer from high latency, reliance on consistent internet, and poor privacy practices.

Primary Objectives:

- **Real-Time Spatial Awareness:** Detect hazards (vehicles, stairs, obstacles) with <500ms latency to ensure user safety.
- **Fault-Tolerant Architecture:** Ensure critical features (SOS, Location) function even without active server connection.
- **Accessible & Intuitive UX:** Design for 'Eyes-Free' operation using high-contrast UI, Haptic feedback, and TalkBack integration.
- **Privacy-First Design:** Process sensitive visual data on user-controlled hardware, minimizing third-party data exposure.

3. System Architecture & Data Flow

The system employs a tightly coupled **Client-Server architecture** communicating via a bidirectional WebSocket protocol over a secure local or reverse-tunneled connection.

Component	Role & Responsibilities
Android Client (The Sensor)	• **CameraX**: Captures 640x480 YUV frames @ 30FPS. • **Packetizer**: Compresses frames + timestamp headers. • **Sensors**: FusedLocationProvider for GPS, Accelerometer. • **Interaction**: TTS (Text-to-Speech) output, SOS triggers.
Transmission Layer (The Bridge)	• **Protocol**: Custom binary protocol (Byte-aligned headers). • **Transport**: Async WebSocket (WSS) with JWT Authentication. • **Optimization**: Sliding window flow control to prevent lag.
Python Backend (The Brain)	• **Orchestrator**: Routes frames to specialized AI engines. • **Inference**: YOLOv8 (Objects), MiDaS (Depth), dlib (Face). • **NLP Router**: Distinguishes 'Help me' from 'Describe scene'.

4. Core Technology Stack & Implementation

Frontend (Android Native):

Built with **Kotlin** and **Jetpack Compose**. Leverages **Hilt** for dependency injection, **Coroutines** for concurrency, and **LifecycleService** for robust background execution.

Backend & AI (Python):

Module	Technology / Model	Purpose
Object Detection	YOLOv8 Nano (Ultralytics)	Real-time identification of 80+ classes (Person, Car, Chair).
Depth Perception	MiDaS Small (Intel)	Monocular depth estimation to gauge distance to obstacles.
Face Recognition	dlib / OpenCV LBPH	Identifying known individuals (Privacy-compliant).
Scene Intelligence	Google Gemini 2.0 Flash	Generative description of complex scenarios and VQA.
OCR	EasyOCR (PyTorch)	Reading signboards, documents, and labels.

Implementation Phases:

- **Phase 1: Foundation.** Setup of WebSocket pipeline, core YOLO inference, and Android CameraX.
- **Phase 2: Visual Intelligence.** Integration of MiDaS Depth, Face Rec, and OCR engines.
- **Phase 3: Safety Nets.** Implementation of NLP Intent Routing, GPS Location, and Emergency SOS SMS.
- **Phase 4: Production.** Latency optimization, Dockerization, and Play Store compliance.

5. Performance & Challenges Solved

1. Bypassing Android Background Limits:
Android 14 restricts background camera usage. We solved this by implementing a **Foreground Service** of type `location` and `camera`, bound to the `LifecycleService`. This ensures AIVA 'sees' even when the phone is in the user's pocket.

2. Latency Engineering:
To achieve <500ms response time, we optimized the network payload by compressing frames to JPEG Q75 and implementing a 'Drop-Oldest' backpressure strategy in the WebSocket buffer. This prevents lag accumulation.

3. Cross-Platform Compatibility:
The `dlib` library caused critical crashes on Windows. We engineered a fallback mechanism using **OpenCV's LBPH Face Recognizer**, ensuring the system remains functional across different server OS environments.

6. Launch Strategy & Future Scope

Beta Launch Strategy:

The application is prepared for a closed Alpha release via the **Google Play Console Internal Testing** track. We have implemented strict Data Safety protocols (Privacy Policy, encrypted storage) to meet Store compliance requirements. Initial rollout targets a focus group of 50 visually impaired users.

Future Roadmap:

➜ **Edge AI Migration:** Porting YOLO and MiDaS to TensorFlow Lite for completely offline inference.

➜ **Wearable Integration:** Logic to support smart glasses (e.g., Ray-Ban Meta) as input devices.

➜ **Haptic Navigation:** Integrating vibration patterns to guide users left/right without audio overload.

7. Conclusion

AIVA represents a paradigm shift in accessible technology. By decoupling the 'Brain' (High-power compute) from the 'Sensor' (Smartphone), we have created a system that is powerful, affordable, and continuously improving. This project is not just a technical demonstration; it is a step towards a world where technology acts as a genuine equalizer, granting independence and dignity to millions.